

Viva Preparation Guide



Web_Technology_Exam_Questions.pdf

PDF



for each questions i want you to give full descriptive answers for viva and help me prepare for each counter questions from that questions and also i want each answers's understanding from the basics

I've mapped the question sheet into sections. I'll start by giving you a viva-ready pattern you can reuse, then cover the first set in a way that builds from the basics, and I'll continue unit by unit.

Your PDF covers oral questions on web servers, application servers, website security, basic web architecture, HTML, JavaScript/DOM, servlets/XML, JSP/web services, server-side scripting, Ruby on Rails, and CSS.

Web_Technology_Exam_Questions

For viva, the safest way to answer is:

1. **Start from the basics** — define the term in one line.
2. **Explain how it works** — mention the flow, components, or process.
3. **Give one example** — this makes the answer sound complete.
4. **Prepare for counters** — know the "difference between", "why", and "how" questions.

Below is a viva-ready guide in that style.

1) Web Server

1. What is a Web Server?

A web server is software that stores web content and delivers it to users when they request a page through a browser. It receives HTTP or HTTPS requests, finds the requested file or runs application code, and sends back a response. Examples are Apache, Nginx, and Microsoft IIS.

Counter questions:

- How is a web server different from a browser?
- What is the difference between static and dynamic content?
- Why do we need HTTP/HTTPS?

2. Explain the working of a web server.

First, the user enters a URL in the browser. The browser sends a request to the server through the network. The web server processes the request, locates the resource, and returns a response such as HTML, CSS, images, or data. If the page is dynamic, the server may call application code before sending the result.

Counter questions:

- What happens after a URL is entered?
- What is the role of DNS here?
- What is an HTTP response?

3. Differentiate between Web Server and Browser.

A browser is client-side software used by the user to request and display web pages. A web server is server-side software that stores and sends web pages or data. The browser sends requests; the server answers them.

Counter questions:

- Which one runs on the client side?
- Which one stores website files?
- Can a browser act as a server?

4. Name some popular web servers.

Common web servers are Apache HTTP Server, Nginx, Microsoft IIS, LiteSpeed, and Tomcat in some web application setups.

Counter questions:

- Which one is open source?
- Which one is from Microsoft?
- Is Tomcat a web server or application server?

5. What is the role of Apache Web Server?

Apache is one of the most widely used web servers. It handles HTTP requests, serves static files, supports modules, allows configuration, and can work with dynamic languages like PHP and Python through integration.

Counter questions:

- Why is Apache popular?
- What are modules in Apache?
- Can Apache serve dynamic pages?

6. What is IIS?

IIS stands for Internet Information Services. It is Microsoft's web server platform used mainly on Windows servers to host websites and web applications.

Counter questions:

- Who developed IIS?
- On which operating system is it commonly used?
- How is IIS different from Apache?

7. How does a web server handle client requests?

The server listens on a port, usually 80 for HTTP or 443 for HTTPS. When a request arrives, it checks the resource path, permissions, and file type. Then it either returns a static file or forwards the request to application logic for dynamic processing.

Counter questions:

- What is port 80 used for?
- What happens if the file is not found?
- What is the status code for "not found"?

8. Explain HTTP request and response process.

HTTP works as a request-response protocol. The browser sends a request message containing a method like GET or POST, headers, and sometimes data. The server sends back a response containing a status code, headers, and the content body.

Counter questions:

- What are GET and POST?
- What is included in headers?
- What does a 200 or 404 status mean?

9. What is virtual hosting?

Virtual hosting means one physical web server can host multiple websites. Each website can have its own domain name and content, even though they share the same server hardware.

Counter questions:

- Why is virtual hosting useful?
- What is name-based virtual hosting?
- Can one server host multiple domains?

10. Difference between static and dynamic content on a web server.

Static content does not change unless a developer edits it, such as HTML files, images, and CSS. Dynamic content is generated at runtime and may change based on user input, database values, or time. Dynamic pages often use server-side programming.

Counter questions:

- Give examples of static and dynamic pages.
- Which one needs a database usually?
- Which one is faster to serve?

11. What is port number 80 used for?

Port 80 is the default port for HTTP traffic. When a browser connects to a website using HTTP, it usually communicates on port 80 unless another port is specified.

Counter questions:

- What is the port for HTTPS?
- Why are ports needed?
- What is the difference between a port and an IP address?

12. What is HTTPS and why is it important?

HTTPS is the secure version of HTTP. It uses SSL/TLS encryption to protect data between browser and server. It is important because it keeps passwords, personal data, and transactions safe from interception.

Counter questions:

- What is SSL/TLS?
- How does HTTPS protect data?
- Why is HTTPS better than HTTP?

13. What is the role of DNS in web hosting?

DNS translates human-readable domain names like **example.com** into IP addresses that computers understand. Without DNS, users would have to remember numerical IP addresses for each website.

Counter questions:

- What is a domain name?
- What is an IP address?

- What happens when DNS fails?

14. What is load balancing in web servers?

Load balancing is the distribution of client requests across multiple servers so no single server becomes overloaded. It improves speed, reliability, and availability.

Counter questions:

- Why is load balancing used?
- What happens if one server fails?
- Is load balancing related to scalability?

15. What are the advantages of web servers?

Web servers make websites accessible over a network, support multiple users, manage requests efficiently, and allow hosting of both static and dynamic content. They also improve centralized management and security control.

Counter questions:

- Why can't we just open HTML files directly for websites?
 - What is the benefit of central hosting?
 - How do servers support many users at once?
-

2) Application Server

1. What is an Application Server?

An application server is software that runs business logic and dynamic web applications. It goes beyond serving files and provides services like transactions, sessions, security, and database access.

Counter questions:

- How is it different from a web server?
- Why do enterprise apps need it?
- What services does it provide?

2. Differentiate between Web Server and Application Server.

A web server mainly handles HTTP requests and serves static content. An application server runs application logic and provides enterprise features such as transactions, messaging, and session management.

Counter questions:

- Can a web server run business logic?
- Can an application server also serve web pages?
- Which one is better for dynamic apps?

3. Give examples of Application Servers.

Examples include Apache Tomcat, JBoss/WildFly, GlassFish, WebLogic, and WebSphere.

Counter questions:

- Which one is open source?
- Is Tomcat a full application server?
- Which ones are used in enterprise systems?

4. Explain the role of middleware in application servers.

Middleware is the software layer between the operating system and the application. In application servers, it helps in communication, security, transaction control, and integration with databases or other systems.

Counter questions:

- Why is middleware needed?
- Is middleware the same as a framework?
- What kind of services does it provide?

5. What services are provided by an application server?

Application servers provide session management, transaction processing, security, connection pooling, clustering, and integration with databases and messaging systems.

Counter questions:

- What is connection pooling?
- What is clustering?
- What is transaction management?

6. What is session management?

Session management means keeping track of a user's state across multiple requests. Since HTTP is stateless, sessions help the server remember a user after login or during shopping cart usage.

Counter questions:

- Why is HTTP stateless?
- What is the difference between session and cookie?
- How does a server identify a user session?

7. Explain client-server architecture.

In client-server architecture, the client requests a service and the server provides it. The client is usually the browser or app, and the server stores data or performs processing.

Counter questions:

- Which side initiates the request?
- Is the browser a client?
- Give a real-world example.

8. What is a servlet container?

A servlet container is the runtime environment that manages Java servlets. It handles servlet life cycle, request mapping, session management, and communication with the web server.

Counter questions:

- What is a servlet?
- Which container is commonly used with Tomcat?
- What does the container do for a servlet?

9. What is deployment in an application server?

Deployment means packaging and installing an application on the server so it can run. This usually includes configuration files, code files, and resources.

Counter questions:

- What is a WAR file?
- Why do we deploy applications?
- Can deployment be automated?

10. Explain scalability in application servers.

Scalability is the ability to handle more users or more work without major performance loss. Application servers achieve it through clustering, load balancing, and resource management.

Counter questions:

- What is horizontal scaling?

- What is vertical scaling?
- Why is scalability important?

11. What is connection pooling?

Connection pooling reuses database connections instead of creating a new one every time. This saves time and improves performance.

Counter questions:

- Why is opening a DB connection expensive?
- How does pooling improve speed?
- Is it useful in high-traffic apps?

12. Role of an application server in enterprise applications.

It handles core business logic, security, transactions, messaging, and integration with multiple enterprise systems. It makes large applications easier to manage and scale.

Counter questions:

- Why not put all logic in the web server?
- What is enterprise application?
- Why is transaction support important?

13. Explain transaction management.

Transaction management ensures a set of operations either all succeed or all fail together. This keeps data consistent in systems like banking or e-commerce.

Counter questions:

- What does ACID mean?
- Why is rollback needed?
- Give an example of a transaction.

14. What is clustering in application servers?

Clustering means running multiple servers together as one logical unit. It improves fault tolerance and load sharing.

Counter questions:

- How does clustering improve reliability?
- What happens if one node fails?

- Is clustering related to high availability?

15. Why are application servers important for dynamic websites?

Dynamic websites need business logic, sessions, database communication, and security. Application servers provide the environment needed to handle all of that efficiently.

Counter questions:

- Why are static sites different?
 - What makes a site dynamic?
 - Can dynamic content work without an application server?
-

3) Website Security

1. Why is website security important?

Website security protects user data, prevents unauthorized access, and builds trust. Without it, a site can be hacked, defaced, or used to steal information.

Counter questions:

- What can happen if a site is insecure?
- Why is user trust important?
- Is security only for large websites?

2. What is SSL/TLS certificate?

It is a digital certificate used to establish secure, encrypted communication between browser and server. It also helps verify the identity of the website.

Counter questions:

- What does TLS do?
- Why do websites need certificates?
- What is the difference between HTTP and HTTPS?

3. Explain HTTPS protocol.

HTTPS is HTTP over TLS. It encrypts the communication so attackers cannot easily read or change the data in transit.

Counter questions:

- What layer provides encryption?

- Why is HTTPS mandatory for login pages?
- What does the lock symbol mean in browsers?

4. What is authentication and authorization?

Authentication checks who the user is, such as verifying username and password. Authorization checks what the authenticated user is allowed to do.

Counter questions:

- Which comes first?
- Is login authentication or authorization?
- Give an example of authorization.

5. What is a firewall?

A firewall is a security system that monitors and controls network traffic based on rules. It blocks suspicious or unauthorized access.

Counter questions:

- Is a firewall hardware or software?
- What does it protect against?
- How is it different from antivirus?

6. What is SQL Injection attack?

SQL injection is a hacking technique where malicious SQL code is inserted into input fields to manipulate the database. It can expose, delete, or modify data.

Counter questions:

- Why is input validation important?
- How can parameterized queries help?
- Give an example of a vulnerable form.

7. What is Cross Site Scripting (XSS)?

XSS is an attack where malicious scripts are injected into web pages and executed in users' browsers. It can steal cookies, sessions, or data.

Counter questions:

- What is the danger of XSS?
- How can output encoding prevent it?

- What is the difference between XSS and SQL injection?

8. How can passwords be stored securely?

Passwords should be hashed with a strong hashing algorithm and salted. They should never be stored in plain text.

Counter questions:

- What is hashing?
- Why use salt?
- Why is encryption alone not enough for passwords?

9. What is encryption?

Encryption converts readable data into unreadable form using a key so only authorized parties can decode it. It protects data in transit and sometimes at rest.

Counter questions:

- What is the difference between encryption and hashing?
- What is symmetric encryption?
- Why is encryption needed in web security?

10. What is two-factor authentication?

Two-factor authentication adds a second verification step besides the password, such as OTP, authenticator app, or biometric check. It improves account security.

Counter questions:

- Why is 2FA safer than password alone?
- What are common second factors?
- Is OTP enough by itself?

11. What are cookies and sessions?

Cookies are small pieces of data stored in the browser. Sessions are server-side records used to maintain user state; cookies often store the session ID.

Counter questions:

- What is stored on client side and what is stored on server side?
- Why do we need sessions?
- Can cookies be disabled?

12. How can websites be protected from hackers?

Use secure coding, HTTPS, strong authentication, regular updates, input validation, access control, backups, firewalls, and monitoring. Security is a combination of many layers, not one tool.

Counter questions:

- Why is secure coding important?
- What is input validation?
- Why are updates necessary?

13. What is CAPTCHA and why is it used?

CAPTCHA is a test used to tell humans and bots apart. It helps stop spam, brute-force attacks, and automated abuse.

Counter questions:

- Why do websites use CAPTCHA on forms?
- Does CAPTCHA guarantee security?
- Can bots still bypass it?

14. Explain the importance of regular software updates.

Updates patch security holes, fix bugs, and improve performance. Old software often becomes easy to attack because known vulnerabilities remain unpatched.

Counter questions:

- Why are old versions risky?
- What is a security patch?
- Should server software also be updated regularly?

15. What is data backup and disaster recovery?

Backup means keeping copies of important data. Disaster recovery is the process of restoring systems and data after failure, attack, or loss.

Counter questions:

- Why are backups necessary?
- How often should backups be taken?
- What happens if there is no recovery plan?

16. What are secure coding practices?

They are coding methods that reduce vulnerabilities, such as input validation, output encoding, avoiding hardcoded passwords, using prepared statements, and checking permissions.

Counter questions:

- Why is validation important?
- What are prepared statements?
- Why should secrets not be hardcoded?

17. How does antivirus software help website security?

Antivirus detects and removes malicious software on systems. It can help protect servers or admin machines from malware infection.

Counter questions:

- Is antivirus enough for web security?
- Does it stop SQL injection?
- Where is antivirus most useful?

18. What is phishing?

Phishing is a fraud technique where attackers trick users into revealing passwords or sensitive data through fake websites, emails, or messages.

Counter questions:

- How can users identify phishing?
- Why is phishing dangerous?
- What is the role of awareness training?

19. Explain the role of access control in website security.

Access control decides who can access which resource and what actions they can perform. It prevents unauthorized users from reaching restricted data or functions.

Counter questions:

- What is role-based access control?
- How is access control different from authentication?
- Why is least privilege important?

20. What are the best practices to make a website secure?

Use HTTPS, strong passwords, 2FA, secure coding, regular updates, backups, logging, access control, input validation, and protection against common attacks like XSS and SQL injection.

Counter questions:

- Which attack is most common in web apps?
 - Why are backups part of security?
 - Why is HTTPS not enough alone?
-

4) Basic Web Architecture

1. Define Web Architecture.

Web architecture is the overall design of how web clients, servers, protocols, databases, and other components work together to deliver web applications.

Counter questions:

- What are the main components?
- Is it only about UI and backend?
- Why is architecture important?

2. Differentiate between Internet and Intranet.

The Internet is a public global network accessible to anyone. An intranet is a private internal network used inside an organization.

Counter questions:

- Which one is private?
- Can an intranet use web technologies?
- What is an extranet?

3. Explain the role of a web browser.

A browser is a client application that requests web pages, interprets HTML/CSS/JavaScript, and displays content to the user.

Counter questions:

- What languages does a browser understand?
- Is browser only for HTML?
- Name some browsers.

4. What is a web server? Give examples.

A web server is software that responds to HTTP requests and serves web content. Examples include Apache, Nginx, and IIS.

Counter questions:

- What does a web server do?
- Is Tomcat a web server?
- Which protocol does it use?

5. Explain the client-server model.

The client sends a request, and the server responds with data or services. This model is the foundation of the web.

Counter questions:

- Who initiates communication?
- What is a request-response model?
- Give a website example.

6. Define HTTP and HTTPS.

HTTP is the protocol used for communication between browser and web server. HTTPS is the secure version of HTTP that uses TLS encryption.

Counter questions:

- What is the difference between them?
- Which port numbers are used?
- Why is HTTPS preferred?

7. What is DNS? Explain its working.

DNS maps domain names to IP addresses. When a browser asks for a site, DNS translates the name into the server's address so the request can reach the correct machine.

Counter questions:

- Why is DNS called the internet's phonebook?
- What happens in DNS lookup?
- What is an IP address?

8. Explain the structure of a URL.

A URL contains the scheme, domain name, path, and sometimes query parameters. Example: in `https://example.com/shop?id=10`, `https` is the scheme, `example.com` is the domain, `/shop` is the path, and `id=10` is the query string.

Counter questions:

- What is the difference between URL and domain name?
- What is a query parameter?
- Why is the scheme important?

9. Differentiate between static and dynamic web pages.

Static pages show the same content to every user unless edited manually. Dynamic pages change according to user input, database content, or application logic.

Counter questions:

- Which one needs server-side code?
- Give examples of both.
- Which one is simpler to build?

10. Explain the working of TCP/IP protocol suite.

TCP/IP is a set of protocols used for communication over networks. IP handles addressing and routing, while TCP ensures reliable data transfer by breaking data into packets and reassembling it.

Counter questions:

- What is the role of IP?
- Why do we need TCP?
- How does packet delivery happen?

11. Explain Basic Web Architecture with neat diagram.

At a basic level, the browser sends a request through the Internet to a web server, which may access a database and return a response. In simple form: **Browser** → **Web Server** → **Application/Database** → **Web Server** → **Browser**.

Counter questions:

- What are the layers in web architecture?
- Where does the database fit?
- What is the role of the browser?

12. Describe the client-server communication process in detail.

The client enters a URL, DNS resolves the domain, the browser establishes a connection, sends an HTTP request, and the server processes it. The response returns with status codes, headers, and body content, which the browser renders.

Counter questions:

- What happens before the request is sent?
- Why do we need DNS?
- What does the browser do with the response?

13. Explain the working of HTTP request and response cycle.

The browser sends a request with method, headers, and optionally data. The server processes it and sends a response with a status code such as 200, 404, or 500, plus the content body.

Counter questions:

- What is the meaning of status codes?
- What is the difference between GET and POST?
- What is in response headers?

14. Discuss various Internet protocols used in web technologies.

Common protocols include HTTP/HTTPS for web communication, TCP/IP for networking, DNS for name resolution, FTP for file transfer, SMTP for email, and TLS for security.

Counter questions:

- Which protocol is used for secure browsing?
- What does DNS do?
- What is TCP/IP?

15. Explain the architecture and functions of web servers and browsers.

The browser is the client that displays content and sends requests. The web server receives requests, serves files or runs server logic, and returns responses. Together they form the front-end communication path of the web.

Counter questions:

- Which one is on the user side?
- Which one stores web content?
- Can they work without each other?

16. Compare static websites and dynamic websites with examples.

Static websites are fixed and simple, such as a personal portfolio. Dynamic websites change based on database or user interaction, such as online shopping sites or social media platforms.

Counter questions:

- Which one is easier to host?
- Which one supports login and carts?
- Which one is more scalable for content changes?

17. Explain DNS architecture and domain name resolution process.

The browser asks a DNS resolver for the IP of a domain. The resolver checks cached data or queries DNS servers until it finds the correct IP, then returns it to the browser.

Counter questions:

- What is DNS caching?
- What is a resolver?
- Why is DNS resolution important?

18. Discuss advantages and limitations of web-based applications.

Advantages include accessibility from anywhere, central updates, and easier maintenance. Limitations include dependence on internet connectivity, browser compatibility issues, and security concerns.

Counter questions:

- Why are web apps popular?
- What is their biggest limitation?
- How do browsers affect web apps?

19. Explain three-tier web architecture with diagram.

Three-tier architecture has a presentation layer, application layer, and data layer. The browser handles presentation, the application server handles logic, and the database stores data.

Counter questions:

- What is the role of each tier?
- Why is three-tier better than one-tier?

- What is the data layer?

20. Describe the components of web architecture and their interactions.

The main components are browser, web server, application server, database, DNS, and network protocols. They work together in request-response flow to deliver web content securely and efficiently.

Counter questions:

- What is the role of each component?
- Where does security fit in?
- Why is the architecture layered?

5) HTML

1. Define HTML and explain its features.

HTML stands for HyperText Markup Language. It is used to structure web content using tags like headings, paragraphs, lists, links, images, and forms. It is not a programming language; it is a markup language.

Counter questions:

- What does HTML stand for?
- Is HTML a programming language?
- Why is HTML called markup?

2. Differentiate between ordered and unordered lists.

Ordered lists use numbers or letters and show sequence. Unordered lists use bullets and are used when order does not matter.

Counter questions:

- Which tag is used for ordered list?
- Which tag is used for unordered list?
- When should each be used?

3. Explain semantic HTML tags with examples.

Semantic tags clearly describe the meaning of content, such as `<header>`, `<nav>`, `<article>`, and `<footer>`. They improve readability, accessibility, and SEO.

Counter questions:

- Why are semantic tags better than only `<div>`?
- Give examples of semantic elements.
- How do they help SEO?

4. Write HTML code to create a student registration form.

A registration form should include inputs like name, email, password, gender, course, date of birth, and submit button. It uses `<form>`, `<input>`, `<select>`, `<textarea>`, and labels for user-friendly input.

Counter questions:

- What is the purpose of labels?
- Which input type is used for password?
- Why is form validation needed?

5. Explain the structure of an HTML document.

An HTML document usually contains `<!DOCTYPE html>`, `<html>`, `<head>`, and `<body>`. The head contains metadata and the body contains visible page content.

Counter questions:

- What goes inside `<head>`?
- What is the purpose of `DOCTYPE`?
- What is the role of `<body>`?

6. Design a webpage using tables, lists, images, and hyperlinks.

A webpage can use tables to organize rows and columns, lists for grouping items, images for visuals, and hyperlinks to connect pages or websites. All of these are basic HTML building blocks.

Counter questions:

- When should tables be used?
- What tag adds an image?
- What tag creates a hyperlink?

7. Explain HTML forms and form controls with suitable examples.

Forms collect user input. Form controls include text boxes, radio buttons, checkboxes, dropdowns, textareas, and submit buttons.

Counter questions:

- Why are forms important?
- What is the difference between radio and checkbox?
- What does `method` in form mean?

8. Discuss multimedia elements in HTML5.

HTML5 supports audio, video, and embedded media without external plugins. The main tags are `<audio>` and `<video>`.

Counter questions:

- Why was HTML5 important?
- What tags are used for audio and video?
- What is an advantage of native multimedia support?

9. Explain different types of CSS with examples.

Inline CSS is written inside an element, internal CSS is written in the `<style>` tag, and external CSS is stored in a separate file. External CSS is generally preferred for larger projects.

Counter questions:

- Which CSS type has the highest priority?
- Why is external CSS preferred?
- What is the difference between internal and external CSS?

6) JavaScript and DOM

1. Explain JavaScript data types.

JavaScript data types include primitive types like string, number, boolean, null, undefined, bigint, and symbol, plus non-primitive objects. Data types decide what kind of values variables can hold.

Counter questions:

- What is the difference between primitive and non-primitive types?
- What is `null` versus `undefined`?
- Is an array a data type?

2. What is DOM? Explain its importance.

DOM stands for Document Object Model. It is the tree-like structure of a webpage that JavaScript can read and change dynamically.

Counter questions:

- Why is DOM important?
- How does JavaScript use DOM?
- What is a DOM tree?

3. Write a JavaScript program for form validation.

Form validation checks user input before submission, such as ensuring fields are not empty, email format is correct, and password length is sufficient. This prevents bad data from reaching the server.

Counter questions:

- Why validate on the client side?
- Is client-side validation enough?
- What is the role of server-side validation?

4. Explain JavaScript events with examples.

Events are actions like click, mouseover, keypress, submit, and load that trigger JavaScript code. They let web pages respond to user interaction.

Counter questions:

- What is an event handler?
- Give common events.
- What happens on a click event?

5. Differentiate between == and ===.

== compares values after type conversion, while === compares both value and type without conversion. === is safer and preferred.

Counter questions:

- Why is === preferred?
- What is type coercion?
- Give an example where == behaves unexpectedly.

6. Explain DOM manipulation methods with examples.

DOM manipulation means changing HTML elements using JavaScript. Common methods include `getElementById`, `querySelector`, `createElement`, `appendChild`, and changing `innerHTML` or styles.

Counter questions:

- What is the difference between `querySelector` and `getElementById`?
- How do you add a new element?
- Can JavaScript change CSS through DOM?

7. Write a JavaScript program to perform calculator operations.

A calculator program takes input numbers and an operator, then performs addition, subtraction, multiplication, or division using conditional statements or functions.

Counter questions:

- How do you handle divide by zero?
- Why use functions in the calculator?
- Can events be used in the calculator?

8. Explain event handling in JavaScript.

Event handling means attaching code to an event so a page reacts when the event happens. This is done using event listeners or event properties.

Counter questions:

- What is `addEventListener`?
- Why is event handling useful?
- What is event bubbling?

9. Discuss advantages and limitations of JavaScript.

JavaScript makes pages interactive, validates forms, controls DOM, and runs in the browser. Limitations include security exposure on the client side and browser differences, though modern JavaScript is powerful.

Counter questions:

- Why is JavaScript essential for web pages?
- Can it access the server directly?
- Why should sensitive checks not rely only on JavaScript?

7) Servlets and XML

1. Explain servlet life cycle.

The servlet life cycle has three main stages: initialization, request handling, and destruction. The servlet container creates the servlet object, calls `init()`, then `service()` or `doGet()/doPost()` for requests, and finally `destroy()` when it is removed.

Counter questions:

- Which method is called once?
- Which method handles requests?
- What is the role of the container?

2. Differentiate between GET and POST methods.

GET sends data in the URL and is mainly used to fetch data. POST sends data in the request body and is better for submitting forms and sensitive information.

Counter questions:

- Which method is safer for passwords?
- Which one can be bookmarked easily?
- Which one has URL length limits?

3. Explain XML syntax rules.

XML uses user-defined tags and strict syntax. Tags must be properly nested, case-sensitive, and closed correctly, and there should be only one root element.

Counter questions:

- What is the root element?
- Is XML case-sensitive?
- How is XML different from HTML?

4. What are cookies and sessions?

Cookies store small data in the browser, while sessions store user state on the server. They are both used to maintain continuity in a stateless HTTP environment.

Counter questions:

- Which one is client-side?
- Which one is more secure?
- How is a session created?

5. Explain DOM parser and SAX parser.

A DOM parser loads the full XML document into memory and allows random access. A SAX parser reads XML sequentially and is more memory efficient.

Counter questions:

- Which parser is faster for large files?
- Which parser uses less memory?
- Which parser allows editing the full tree easily?

6. Explain servlet architecture with diagram.

A browser sends a request to the web server or servlet container. The container maps the request to the correct servlet, calls the appropriate method, and sends the response back to the browser.

Counter questions:

- What is the role of the servlet container?
- What is request mapping?
- How is response generated?

7. Write a servlet program for user login validation.

A login servlet receives username and password from a form, checks them against stored values or a database, and returns success or failure. This is a common example of request handling.

Counter questions:

- Why is validation important?
- Should passwords be stored in plain text?
- Can a servlet connect to a database?

8. Explain XML DTD with example.

DTD stands for Document Type Definition. It defines the legal structure of an XML document, including allowed elements and attributes.

Counter questions:

- Why use DTD?
- What does it validate?
- Is DTD the only validation method?

9. Discuss session management techniques in servlets.

Session tracking in servlets can be done using cookies, URL rewriting, hidden fields, and `HttpSession`. `HttpSession` is commonly used because it is convenient and server-side.

Counter questions:

- Which technique is most common?
 - What is URL rewriting?
 - What is the risk of cookies being disabled?
-

8) JSP and Web Services

1. Explain JSP implicit objects.

JSP implicit objects are pre-defined objects available in JSP pages, such as `request`, `response`, `out`, `session`, `application`, `config`, `pageContext`, `page`, and `exception`.

Counter questions:

- Why are they called implicit?
- Which object is used for session?
- What does `out` do?

2. Differentiate between Servlets and JSP.

Servlets are Java classes used mainly for business logic and control flow. JSP is better for presentation because it allows HTML mixed with Java code, though modern practice minimizes scriptlets.

Counter questions:

- Which one is better for UI?
- Which one is compiled into servlet?
- Why is MVC used with these?

3. What are Web Services?

Web services are software systems that enable communication between applications over a network using standard protocols. They allow different systems to exchange data regardless of language or platform.

Counter questions:

- Why are web services useful?
- What are SOAP and REST?
- What format is commonly used in REST?

4. Explain REST architecture.

REST is an architectural style that uses HTTP methods like GET, POST, PUT, and DELETE to work with resources identified by URLs. It is lightweight and widely used in APIs.

Counter questions:

- What is a resource in REST?
- Why is REST popular?
- How does REST use HTTP methods?

5. Write advantages of JSP.

JSP makes dynamic web page creation easier, separates presentation from business logic, and integrates with Java features. It is useful for building server-generated HTML pages.

Counter questions:

- Why is JSP easier for UI than servlet?
- What is the role of JSP in MVC?
- Is JSP still used in modern Java web apps?

6. Explain JSP life cycle with diagram.

The JSP life cycle includes translation into servlet, compilation, loading, initialization, request handling, and destruction. In simple terms, JSP is first converted to a servlet before execution.

Counter questions:

- Why is JSP converted to servlet?
- Which method corresponds to initialization?
- What is the first stage in JSP life cycle?

7. Develop a JSP application for database connectivity.

A JSP app can take input, send it to a servlet or script, connect to a database through JDBC, execute queries, and display results on the page. The key idea is connecting web UI with database backend.

Counter questions:

- What is JDBC?
- Why use servlet in front of database logic?
- Is direct DB code in JSP a good practice?

8. Compare SOAP and REST Web Services.

SOAP is a protocol with strict rules and uses XML, while REST is an architectural style that is simpler and usually uses JSON or XML over HTTP. REST is lighter and more common for modern APIs.

Counter questions:

- Which one is heavier?
- Which one is more flexible?
- What data format does REST often use?

9. Explain MVC architecture in web applications.

MVC stands for Model-View-Controller. Model handles data, View shows the UI, and Controller manages user requests and coordination.

Counter questions:

- Why is MVC useful?
- Which part handles database data?
- Which part shows the webpage?

9) Server-Side Scripting / PHP

1. Explain server-side scripting.

Server-side scripting means code runs on the server, not in the browser. It processes requests, interacts with databases, and generates dynamic output.

Counter questions:

- What is the difference between client-side and server-side scripting?
- Why is server-side scripting needed?
- Give examples of server-side languages.

2. What are sessions and cookies?

Cookies store small information in the browser, while sessions store user data on the server for a limited time. They are used for login and user tracking.

Counter questions:

- Which one is more secure?
- What is session timeout?
- Can cookies store large data?

3. Explain CRUD operations.

CRUD stands for Create, Read, Update, and Delete. These are the four basic operations used to manage data in databases and applications.

Counter questions:

- Which SQL operations match CRUD?
- Why are CRUD operations important?
- Give an application example.

4. Write advantages of PHP.

PHP is easy to learn, widely used for web development, works well with databases, and is supported by many hosting providers. It is especially useful for server-side scripting.

Counter questions:

- Why is PHP popular?
- Is PHP client-side or server-side?
- What type of websites use PHP?

5. Explain file handling in server-side scripting.

File handling means creating, reading, writing, and deleting files on the server through code. It is used for logs, uploads, reports, and configuration.

Counter questions:

- Why is file handling useful?
- What precautions are needed with uploads?
- Can a script read server files directly?

6. Write a PHP program for user authentication.

A PHP authentication program receives username and password, checks them against stored records, and allows or denies access. It may also create a session after successful login.

Counter questions:

- Why should passwords be hashed?
- What is session-based login?
- Why is input validation important?

7. Explain database connectivity in PHP.

PHP connects to databases using extensions like MySQLi or PDO. It can send queries, fetch results, and display them in web pages.

Counter questions:

- Why use PDO?
- What is the role of prepared statements?
- Can PHP interact with MySQL?

8. Discuss advantages and disadvantages of server-side scripting.

Advantages include dynamic content, centralized control, database access, and improved security for business logic. Disadvantages include server load, need for backend setup, and possible slower response than pure static content.

Counter questions:

- Why is server-side scripting heavier than static files?
- Why is it still needed?
- Can it work without a database?

9. Develop a CRUD-based web application.

A CRUD app allows users to add, view, edit, and delete records. It usually uses HTML forms, server-side code, and a database.

Counter questions:

- What are the four CRUD actions?
- Which part talks to the database?
- What is an example of a CRUD app?

10) Ruby and Rails

1. Explain features of Ruby.

Ruby is a dynamic, object-oriented language known for simplicity and readability. It supports clean syntax, flexibility, and powerful blocks.

Counter questions:

- Is Ruby object-oriented?
- Why is Ruby easy to read?
- What are blocks?

2. What is Ruby on Rails?

Ruby on Rails is a web application framework built using Ruby. It follows convention over configuration and helps build web apps quickly.

Counter questions:

- What is a framework?
- Why is Rails popular?
- Is Rails the same as Ruby?

3. Explain MVC architecture in Rails.

Rails follows Model-View-Controller architecture. Models manage data, views manage presentation, and controllers manage requests and flow.

Counter questions:

- Why does Rails use MVC?
- Which part handles database interaction?
- What does the controller do?

4. Differentiate between Ruby and Rails.

Ruby is a programming language. Rails is a framework built on Ruby for web development.

Counter questions:

- Can Rails exist without Ruby?
- Is Ruby a framework?
- What is the use of Rails?

5. Explain routing in Rails.

Routing maps incoming URLs to controller actions. It decides which code should run for a given request.

Counter questions:

- Why is routing necessary?
- What happens when a URL is requested?
- What does a controller action mean?

6. Explain Rails framework architecture with diagram.

Rails uses MVC plus routing and Active Record. The request comes in, routing sends it to controller, controller interacts with model, and view generates the output.

Counter questions:

- What is Active Record?
- How does a request reach the view?
- Why is MVC useful in Rails?

7. Develop a simple Rails web application.

A simple Rails app is created by generating a project, defining routes, creating controllers and views, and optionally connecting models to a database.

Counter questions:

- What is the first step in a Rails app?
- What are routes used for?
- What is a view file?

8. Discuss advantages of Ruby on Rails.

Rails allows fast development, clean structure, MVC support, and strong community support. It is good for building standard web applications quickly.

Counter questions:

- Why is Rails called productive?
- What does convention over configuration mean?
- Is Rails suitable for all projects?

9. Explain Active Record in Rails with examples.

Active Record is Rails' ORM layer that links database tables to Ruby classes. It lets developers work with database records as objects.

Counter questions:

- What is ORM?
- Why is Active Record useful?
- How does it simplify database work?

11) CSS

1. What is CSS?

CSS stands for Cascading Style Sheets. It is used to control the appearance of HTML pages, including layout, colors, fonts, spacing, and responsiveness.

Counter questions:

- What does CSS stand for?
- Why is CSS used?
- Can HTML style itself?

2. Why is CSS used in web development?

CSS separates content from presentation, making pages easier to design, maintain, and reuse. It improves visual quality and user experience.

Counter questions:

- Why separate HTML and CSS?
- What is the benefit of reuse?
- How does CSS improve UI?

3. What are the advantages of CSS?

CSS makes pages consistent, reduces repetition, improves loading, and supports responsive design. It also makes maintenance easier.

Counter questions:

- Why is external CSS useful?
- How does CSS improve consistency?
- What is responsive design?

4. Differentiate between HTML and CSS.

HTML defines the structure and content of a webpage. CSS defines the look and styling.

Counter questions:

- Which one creates the page structure?
- Which one controls design?
- Can a page work without CSS?

5. What are the different types of CSS?

The three main types are inline CSS, internal CSS, and external CSS.

Counter questions:

- Which type is best for large websites?
- Which type has highest priority?
- What is the difference between internal and external CSS?

6. Explain Inline CSS with example.

Inline CSS is written directly inside an HTML element using the `style` attribute. It affects only that one element.

Counter questions:

- Is inline CSS reusable?
- Why is it not preferred for large projects?
- Which has higher priority, inline or external?

7. Explain Internal CSS with example.

Internal CSS is written inside a `<style>` tag within the HTML document head. It applies to that page only.

Counter questions:

- When is internal CSS useful?
- Is it better than inline CSS?
- Does it work across multiple pages?

8. Explain External CSS with example.

External CSS is written in a separate `.css` file and linked to HTML using `<link>`. It is the most maintainable option for bigger websites.

Counter questions:

- Why is external CSS preferred?
- Can one CSS file style many pages?
- Does browser cache external CSS?

9. Which type of CSS has highest priority?

Inline CSS generally has the highest priority among inline, internal, and external styles, unless overridden by `!important` or higher specificity rules.

Counter questions:

- What is CSS specificity?
- What does **!important** do?
- Why does inline CSS override external CSS?

10. What is a CSS selector?

A selector is a pattern used to target HTML elements for styling. Examples include element selectors, class selectors, and ID selectors.

Counter questions:

- Why are selectors needed?
- Name different selectors.
- Which selector is most specific?

11. Explain element selector with example.

An element selector targets all elements of a specific tag, such as all `<p>` tags.

Counter questions:

- What elements does it affect?
- Is it broad or specific?
- Give another example.

12. What is an ID selector?

An ID selector targets a single element with a unique ID using `#idname`.

Counter questions:

- Can one ID be used twice?
- Why is an ID selector specific?
- When should an ID be used?

13. What is a class selector?

A class selector targets all elements with the same class name using `.classname`.

Counter questions:

- Can multiple elements share one class?
- How is class different from ID?
- Why are classes useful?

14. Differentiate between class selector and ID selector.

A class is reusable for multiple elements, while an ID is meant for one unique element. ID selectors are more specific than class selectors.

Counter questions:

- Which one should be unique?
- Which one is more reusable?
- Which one has higher specificity?

15. What is a universal selector?

The universal selector `*` applies styles to all elements.

Counter questions:

- When is it useful?
- Is it efficient to style everything?
- Can it cause unwanted styling?

16. What is grouping selector in CSS?

Grouping selector lets you apply the same styles to multiple selectors separated by commas.

Counter questions:

- Why use grouping?
- Does it reduce repetition?
- Give an example.

17. Explain pseudo-class selectors.

Pseudo-classes style elements in a particular state, like `:hover`, `:focus`, or `:visited`.

Counter questions:

- What is `:hover` used for?
- Give examples of pseudo-classes.
- How are they different from classes?

18. What are pseudo-elements?

Pseudo-elements style a part of an element, like `::before`, `::after`, or `::first-line`.

Counter questions:

- What part does `::before` affect?
- How is it different from pseudo-class?
- Why are pseudo-elements useful?

19. What is attribute selector in CSS?

An attribute selector styles elements based on attributes and values, such as `input[type="text"]`.

Counter questions:

- Why use attribute selectors?
- Give an example.
- Can they target links or inputs?

20. What is the syntax of CSS?

CSS syntax uses a selector, followed by curly braces containing property-value pairs. Example: `p { color: blue; }`.

Counter questions:

- What is a declaration?
- What is a property?
- What is a value?

21. Explain background properties in CSS.

Background properties control background color, image, repeat, position, and size. They help set the visual backdrop of elements.

Counter questions:

- Can CSS set background images?
- What does `background-size` do?
- What is `background-repeat`?

22. Difference between margin and padding.

Margin is space outside the border of an element. Padding is space inside the border, between content and border.

Counter questions:

- Which one affects outer spacing?
- Which one affects inner spacing?
- Can margin collapse?

23. Explain border properties in CSS.

Border properties control width, style, and color of an element's border. They define the visible outline around content and padding.

Counter questions:

- What are border styles?
- Can borders be rounded?
- What is `border-radius`?

24. What are font properties in CSS?

Font properties control text appearance, such as font-family, size, weight, style, and line height.

Counter questions:

- What does `font-weight` do?
- What is `font-family`?
- Why is font choice important?

25. Explain text formatting properties in CSS.

Text formatting includes alignment, decoration, transformation, spacing, and indentation. It helps control how text is displayed on the page.

Counter questions:

- What is `text-align`?
- What does `text-decoration` do?
- What is `text-transform`?

26. What is opacity in CSS?

Opacity controls how transparent an element is, from 0 fully transparent to 1 fully visible.

Counter questions:

- What does opacity 0 mean?
- Can text and background both become transparent?

- How is opacity different from visibility?

27. What is the use of z-index?

z-index controls stacking order of overlapping elements. Higher values appear on top when elements are positioned.

Counter questions:

- Does z-index work on all elements?
- Why is positioning important here?
- What happens when elements overlap?

28. Explain display property in CSS.

The display property defines how an element behaves in layout, such as **block**, **inline**, **inline-block**, **flex**, or **none**.

Counter questions:

- What does **display: none** do?
- Difference between block and inline?
- What is flex display?

29. Differentiate between block and inline elements.

Block elements take full width and start on a new line. Inline elements take only required width and stay in the same line.

Counter questions:

- Give examples of block elements.
- Give examples of inline elements.
- Can inline elements have width?

30. What is CSS Box Model?

The box model is the way CSS represents elements as content, padding, border, and margin. It is fundamental for understanding spacing and layout.

Counter questions:

- What are the components of the box model?
- Why is it important?
- Which part is inside the border?

31. Explain the components of box model.

The components are content, padding, border, and margin. Content is the actual text or image area, padding surrounds content, border surrounds padding, and margin is outside spacing.

Counter questions:

- Which part is the outermost?
- Which part is the innermost?
- What does padding do?

32. What is content area in box model?

Content area is the part that holds the actual text, image, or other content. It is the core of the box.

Counter questions:

- What surrounds the content area?
- Is padding inside or outside it?
- Why is it important?

33. What is padding in CSS?

Padding is the inner space between content and border. It improves readability and spacing.

Counter questions:

- Is padding inside the border?
- How is padding different from margin?
- Why use padding?

34. What is margin collapse?

Margin collapse happens when vertical margins of block elements combine into one margin instead of adding together. This is a normal CSS behavior.

Counter questions:

- When does margin collapse happen?
- Is it horizontal or vertical margins?
- Why should developers know this?

35. Explain box-sizing property.

box-sizing controls how width and height are calculated. With **border-box**, padding and border are included in the element's total size.

Counter questions:

- Why is **border-box** useful?
- What is the default box-sizing behavior?
- How does it help layout?

36. How can element dimensions be controlled in CSS?

By using width, height, min-width, max-width, min-height, max-height, padding, margin, and box-sizing properties.

Counter questions:

- What controls total size?
- Why use max-width?
- How does box-sizing affect dimensions?

37. What is positioning in CSS?

Positioning controls where an element appears in the document. The main types are static, relative, absolute, fixed, and sticky.

Counter questions:

- Name the positioning types.
- Why is positioning needed?
- What is the default position?

38. Explain static positioning.

Static is the default position. The element follows normal document flow and cannot be moved using top, bottom, left, or right.

Counter questions:

- Is static the default?
- Can offsets work on static?
- What is normal flow?

39. Explain relative positioning.

Relative positioning moves an element relative to its normal position while keeping space reserved in the layout.

Counter questions:

- Does relative positioning remove space?
- What is the difference from absolute?
- Can it be used as a reference for absolute children?

40. Explain absolute positioning.

Absolute positioning removes the element from normal flow and places it relative to the nearest positioned ancestor or the page if none exists.

Counter questions:

- Relative to what is it placed?
- Does it occupy normal layout space?
- What parent property matters here?

41. Explain fixed positioning.

Fixed elements stay in the same place relative to the browser window even when the page scrolls.

Counter questions:

- Give an example of fixed positioning.
- Does it move on scroll?
- Is it removed from normal flow?

42. Explain sticky positioning.

Sticky positioning behaves like relative until a threshold is reached, then it acts like fixed. It is often used for sticky headers.

Counter questions:

- How is sticky different from fixed?
- When does it become sticky?
- Where is it useful?

43. Differentiate between relative and absolute positioning.

Relative keeps the element in normal flow and moves it from its original place. Absolute removes it from flow and positions it relative to the nearest positioned ancestor.

Counter questions:

- Which one reserves space?
- Which one is removed from flow?
- What happens if no positioned parent exists?

44. What is float property in CSS?

Float moves an element to the left or right, allowing text or inline content to wrap around it. It was heavily used for layouts before flexbox and grid.

Counter questions:

- Why was float used earlier?
- Is float best for modern layout?
- What is clearfix?

45. What is clear property?

Clear prevents an element from sitting beside floated elements. It is used to stop wrapping around floats.

Counter questions:

- Why is clear needed?
- What values does clear have?
- How does it relate to float?

46. Explain Flexbox in CSS.

Flexbox is a layout system for arranging items in one direction, either row or column. It is useful for alignment, spacing, and responsive arrangements.

Counter questions:

- Is Flexbox one-dimensional?
- Why use Flexbox?
- What is `justify-content`?

47. What is CSS Grid?

CSS Grid is a layout system for creating two-dimensional layouts using rows and columns. It is powerful for page-level design.

Counter questions:

- Is Grid one-dimensional or two-dimensional?
- What makes Grid different from Flexbox?
- When should Grid be used?

48. Differentiate between Flexbox and Grid.

Flexbox is best for one-dimensional alignment in a row or column. Grid is best for two-dimensional layouts with both rows and columns.

Counter questions:

- Which one is better for page layout?
- Which one is better for aligning items in a line?
- Can they be used together?

49. What are media queries in CSS?

Media queries apply different styles based on screen size or device characteristics. They are the basis of responsive design.

Counter questions:

- Why are media queries important?
- What do they check?
- Give an example use case.

50. What is responsive web design?

Responsive design makes a website adapt to different screen sizes and devices. It uses flexible layouts, media queries, and scalable elements.

Counter questions:

- Why is responsive design needed?
- Which devices benefit from it?
- What techniques are used?

51. Explain CSS transitions.

Transitions make property changes smooth over time instead of instant. They are often used for hover effects and interactive UI.

Counter questions:

- What is the use of transition duration?

- What triggers transitions?
- How is it different from animation?

52. What are CSS animations?

Animations allow elements to change styles over time using keyframes. They can create more complex movement and visual effects than transitions.

Counter questions:

- What are keyframes?
- How is animation different from transition?
- Where are animations used?

53. What is transform property in CSS?

Transform allows scaling, rotating, translating, or skewing elements. It changes how an element is visually displayed.

Counter questions:

- What does `rotate()` do?
- What does `scale()` do?
- Is transform related to animation?

54. Explain hover effect in CSS.

Hover effect changes style when the mouse pointer is over an element. It is commonly used for buttons, links, and cards.

Counter questions:

- Which pseudo-class is used?
- Why is hover useful?
- Does hover work on touch devices the same way?

55. What is the purpose of Bootstrap?

Bootstrap is a front-end framework that provides ready-made CSS classes and responsive components. It helps build attractive websites faster.

Counter questions:

- Why use Bootstrap?
- Is Bootstrap a programming language?

- What does responsive grid mean?

56. What is viewport in responsive design?

The viewport is the visible area of a webpage on a device. The viewport meta tag helps control how content scales on mobile devices.

Counter questions:

- Why is viewport meta tag important?
- What happens without it on mobile?
- What does it control?

57. What are CSS variables?

CSS variables store reusable values like colors, spacing, or sizes. They make code easier to maintain and update.

Counter questions:

- Why use CSS variables?
- How are they declared?
- Can they be reused across the page?

58. What is inheritance in CSS?

Inheritance means some CSS properties are passed from parent to child elements automatically, such as font-related properties. Not all properties inherit by default.

Counter questions:

- Which properties usually inherit?
- Does margin inherit?
- Why is inheritance useful?

59. What are the limitations of CSS?

CSS cannot perform logic like a programming language, and complex layouts or interactions may still require JavaScript. Browser compatibility and specificity issues can also create challenges.

Counter questions:

- Why do we need JavaScript then?
- What is a CSS specificity issue?
- Does CSS support conditional logic?

If you revise these answers in this format, you will be ready for most viva follow-ups because each topic starts from basics and ends with the common counter questions.

I can turn this into a clean unit-wise notes sheet next.



☐
yes

Web Technology Viva Notes (Unit-Wise)

Based on your question sheet

Web_Technology_Exam_Questions

UNIT 1 — WEB SERVER, APPLICATION SERVER & WEB ARCHITECTURE

1. Web Server

Definition

A web server is software that stores website files and sends them to users through HTTP/HTTPS when requested by a browser.

Examples

- Apache
- Nginx
- IIS

Working of Web Server

1. User enters URL.
2. Browser sends HTTP request.
3. Server receives request.
4. Server processes request.
5. Response is sent back.

Static vs Dynamic Content

Static Content	Dynamic Content
Fixed	Changes dynamically
HTML/CSS files	Generated using server-side code
Faster	Requires processing

Important Viva Points

- Port 80 → HTTP
- Port 443 → HTTPS
- HTTPS uses SSL/TLS encryption
- DNS converts domain names into IP addresses

Common Counter Questions

Q. Why is HTTPS secure?

Because it encrypts communication using SSL/TLS.

Q. What happens when a webpage is requested?

DNS lookup → Connection → HTTP request → Server response → Browser rendering.

2. Application Server

Definition

An application server runs business logic and dynamic applications.

Examples

- Tomcat
- JBoss
- GlassFish

Services Provided

- Session management
- Security
- Database connectivity
- Transaction management
- Connection pooling

Web Server vs Application Server

Web Server	Application Server
Handles HTTP requests	Handles business logic
Mainly static content	Dynamic applications
Example: Apache	Example: Tomcat

Important Concepts

Session Management

Maintains user state across multiple requests.

Connection Pooling

Reuses database connections to improve performance.

Clustering

Multiple servers work together for scalability and reliability.

Common Counter Questions

Q. Why is connection pooling needed?

Creating DB connections repeatedly is expensive.

Q. What is scalability?

Ability to handle increasing users/workload.

3. Website Security

Important Security Concepts

Authentication vs Authorization

Authentication	Authorization
Identifies user	Checks permissions
Login process	Access control

HTTPS

Secure version of HTTP using encryption.

SQL Injection

Attack where malicious SQL code is inserted into input fields.

Prevention

- Prepared statements
- Input validation

Cross Site Scripting (XSS)

Injection of malicious JavaScript into webpages.

Prevention

- Output encoding
- Input sanitization

Encryption vs Hashing

Encryption	Hashing
Reversible	One-way
Used for secure communication	Used for passwords

Cookies vs Sessions

Cookies	Sessions
Stored in browser	Stored on server
Less secure	More secure

Two Factor Authentication (2FA)

Requires password + second verification.

Example:

- OTP
- Authenticator app

Best Practices for Website Security

- HTTPS
- Strong passwords
- Regular updates
- Backups
- Firewalls
- Secure coding

4. Basic Web Architecture

Client-Server Model

Client

Browser requesting resources.

Server

Provides requested resources.

HTTP Request-Response Cycle

Flow

1. URL entered
 2. DNS resolves IP
 3. Browser sends request
 4. Server processes
 5. Response returned
-

URL Structure

Example:

`https://example.com/page?id=10`

Part	Meaning
https	Protocol
example.com	Domain
/page	Path
id=10	Query parameter

TCP/IP Basics

TCP

Reliable communication.

IP

Addressing and routing.

Three Tier Architecture

Presentation Layer

UI/Browser

Application Layer

Business logic

Data Layer

Database

UNIT 2 — HTML & CSS

1. HTML Basics

Definition

HTML is a markup language used to structure webpages.

Structure of HTML Document

```
HTML
<!DOCTYPE html>
<html>
<head>
<title>Page</title>
</head>
<body>
Content
</body>
</html>
```

Semantic Tags

Tag	Purpose
<header>	Header section
<nav>	Navigation
<article>	Main content
<footer>	Footer

Advantage

Improves SEO and readability.

Lists

Ordered List

HTML

```
<ol>
<li>Item</li>
</ol>
```

Unordered List

```
HTML
<ul>
<li>Item</li>
</ul>
```

HTML Forms

Common Controls

- Textbox
- Radio button
- Checkbox
- Dropdown
- Textarea

Example

```
HTML
<form>
<input type="text">
<input type="password">
<button>Submit</button>
</form>
```

Multimedia in HTML5

Audio

```
HTML
<audio controls>
```

Video

```
HTML
<video controls>
```

2. CSS Basics

CSS Types

Type	Description
Inline	Inside element
Internal	Inside <style>
External	Separate CSS file

Priority

Inline > Internal > External

Selectors

Element Selector

```
CSS
p {
  color: red;
}
```

Class Selector

```
CSS
.box {
}
```

ID Selector

```
CSS
#header {
}
```

Margin vs Padding

Margin	Padding
Outside border	Inside border

CSS Box Model

1. Content
2. Padding

3. Border
4. Margin

Positioning Types

Type	Description
Static	Default
Relative	Relative to original position
Absolute	Relative to parent
Fixed	Fixed on screen
Sticky	Sticks during scroll

Flexbox vs Grid

Flexbox	Grid
One-dimensional	Two-dimensional
Row OR column	Rows AND columns

Responsive Design

Media Query Example

CSS
`@media screen and (max-width:600px)`

Purpose

Makes websites adapt to different screen sizes.

UNIT 3 — JAVASCRIPT & DOM

JavaScript Basics

Features

- Client-side scripting

- Dynamic webpages
- Event handling
- DOM manipulation

Data Types

Primitive

- String
- Number
- Boolean
- Null
- Undefined

Non-Primitive

- Object
- Array

== VS ===

==	===
Checks value	Checks value + type

DOM

Definition

DOM is the tree representation of an HTML document.

Common Methods

JavaScript

`document.getElementById()`
`document.querySelector()`

Events

Examples

- click
- mouseover
- keypress
- submit

Form Validation Example

```
JavaScript
if(name==""){
alert("Enter name");
}
```

UNIT 4 — SERVLETS, XML & JSP

Servlet Life Cycle

Methods

1. `init()`
2. `service()`
3. `destroy()`

GET vs POST

GET	POST
Data in URL	Data in body
Less secure	More secure

XML Basics

Rules

- Proper nesting
- Closing tags
- One root element

DOM vs SAX Parser

DOM	SAX
Entire file loaded	Sequential reading
More memory	Less memory

JSP

JSP Life Cycle

1. Translation
2. Compilation
3. Initialization
4. Execution
5. Destruction

MVC Architecture

Component	Purpose
Model	Data
View	UI
Controller	Request handling

SOAP vs REST

SOAP	REST
Protocol	Architectural style
XML only	Usually JSON
Heavy	Lightweight

UNIT 5 — SERVER SIDE SCRIPTING & PHP

Server Side Scripting

Definition

Code executed on the server to generate dynamic content.

Languages

- PHP
- Python
- Java
- Ruby

CRUD Operations

Operation	SQL Equivalent
Create	INSERT
Read	SELECT
Update	UPDATE
Delete	DELETE

PHP Sessions

```
PHP
session_start();
```

Database Connectivity

PHP connects to databases using:

- MySQLi
- PDO

UNIT 6 — RUBY ON RAILS

Ruby Features

- Object-oriented
- Simple syntax
- Dynamic language

Ruby vs Rails

Ruby	Rails
Language	Framework

Rails MVC

Model

Handles database.

View

Handles UI.

Controller

Handles requests.

Active Record

Definition

ORM used in Rails.

Purpose

Maps database tables to Ruby objects.

MOST IMPORTANT VIVA DIFFERENCES

HTTP vs HTTPS

HTTP	HTTPS
Not secure	Secure
Port 80	Port 443

Cookie vs Session

Cookie	Session
Client-side	Server-side

GET vs POST

GET	POST
URL data	Body data

Static vs Dynamic Website

Static	Dynamic
Fixed	Changes dynamically

Flexbox vs Grid

Flexbox	Grid
1D	2D

TOP 20 VERY IMPORTANT VIVA QUESTIONS

1. What is a web server?
2. Difference between web server and application server.
3. Explain HTTP request-response cycle.
4. Difference between HTTP and HTTPS.
5. Explain DNS working.
6. Static vs dynamic websites.
7. What is HTML?
8. Explain semantic tags.
9. What is CSS Box Model?

10. Margin vs padding.
11. What is DOM?
12. Difference between == and ===.
13. Explain JavaScript events.
14. Explain servlet life cycle.
15. Difference between GET and POST.
16. What are cookies and sessions?
17. Explain MVC architecture.
18. SOAP vs REST.
19. Explain CRUD operations.
20. Explain responsive web design.